

# Índice

|                                                       |          |
|-------------------------------------------------------|----------|
| <b>VisioTalk — Reporte de Avance</b>                  | <b>1</b> |
| Estado general . . . . .                              | 1        |
| Fase 1 — Fundación y scaffolding . . . . .            | 1        |
| Fase 2 — Lexer (Análisis léxico) . . . . .            | 1        |
| Fase 3 — Parser y AST (Análisis sintáctico) . . . . . | 2        |
| Fase 4 — Análisis semántico . . . . .                 | 3        |
| Fase 5 — Generación de código . . . . .               | 3        |
| Fase 6 — CLI y debugging . . . . .                    | 4        |
| Runtimes . . . . .                                    | 4        |
| Documentación . . . . .                               | 5        |
| Resumen técnico . . . . .                             | 5        |
| Lo que sigue . . . . .                                | 6        |

## VisioTalk — Reporte de Avance

**Fecha:** 22 de mayo de 2026 **Alcance:** Compilador y runtimes — fases implementadas hasta la fecha

---

### Estado general

El proyecto tiene el pipeline de compilación funcionando de punta a punta desde el lexer hasta la impresión del AST. Las fases de análisis semántico y generación de código existen como módulos pero con lógica mínima o placeholder. Los runtimes de escritorio y móvil no han pasado del scaffolding inicial.

El monorepo contiene 6 crates Rust, un runtime desktop en Python y un runtime mobile en React Native. La documentación está al día con PRD, gramática EBNF, plan de implementación y stack tecnológico.

---

### Fase 1 — Fundación y scaffolding

**Estado:** Completo

Se armó la estructura del monorepo con directorios para `crates/`, `runtime-desktop/` y `runtime-mobile/`. El workspace de Rust (`Cargo.toml` raíz) declara 6 miembros y 10 dependencias externas compartidas. Las herramientas de calidad (`rustfmt`, `clippy`, `ruff`, `ESLint`, `Prettier`) están configuradas.

El proyecto React Native se inicializó con TypeScript sobre la plantilla estándar 0.74+. El runtime desktop tiene su `pyproject.toml` listo para uv.

**Archivos creados:** `Cargo.toml`, `rustfmt.toml`, `.gitignore`, `crates/*/Cargo.toml` (6), `runtime-desktop/pyproject.toml`, `runtime-mobile/package.json`

---

### Fase 2 — Lexer (Análisis léxico)

**Estado:** Completo **Crate:** `vt-lexer` | **Archivo:** `crates/vt-lexer/src/lib.rs` (308 líneas)

Usa **logos 0.14** para generar un autómata finito determinista (DFA) a partir de anotaciones declarativas sobre un enum de Rust. No hay máquina de estados escrita a mano.

## Tokens (18 variantes)

| Categoría           | Tokens                                           |
|---------------------|--------------------------------------------------|
| Palabras reservadas | MOD0, DETECTAR                                   |
| Delimitadores       | {, }, :                                          |
| Comparadores        | <, >, <=, >=, ==                                 |
| Unidades            | m, cm, ft, px                                    |
| Literales           | NumberLit(f64), Ident(String), StringLit(String) |

Los espacios en blanco y comentarios (//) se descartan mediante `#[logos(skip ...)]`. La ambigüedad entre `Ident` y palabras reservadas se resuelve por prioridad implícita de `#[token(...)]` sobre `#[regex(...)]`.

### API pública

```
pub fn tokenize(source: &str) -> Vec<Token>
pub fn tokenize_with_locations(source: &str) -> Vec<(usize, Token, usize)>
```

Ambas filtran errores de logos con `.flatten()`. La segunda incluye offsets de byte (inicio, fin) para reporte de errores con posición.

### Tests

22 pruebas unitarias que cubren: palabras reservadas, delimitadores, comparadores, unidades, literales numéricos y de string, whitespace, comentarios y entrada vacía. Todas pasan.

---

## Fase 3 — Parser y AST (Análisis sintáctico)

**Estado:** Completo **Crates:** vt-parser, vt-core

### AST (vt-core)

5 structs y 3 enums que modelan el árbol sintáctico. Todo deriva `Serialize + Deserialize` para salida JSON/TOML.

```
Program → ModeDecl? + Vec<DetectRule>
DetectRule → label + Vec<Attribute>
Attribute → Distance(DistanceExpr) | Alert(String) | Priority(u32)
DistanceExpr → comparator + value + unit
```

### Parser (vt-parser)

Usa **LALRPOP 0.20** como generador LR(1). La gramática está en `crates/vt-parser/src/grammar.lalrpop` (78 líneas). El `build.rs` invoca `lalrpop::process_root()` para compilar la gramática a Rust en tiempo de build.

El dispatch de atributos (distancia, alerta, prioridad) se hace por primer token: si el parser ve un `Comparator`, construye `Distance`; si ve `StringLit`, `Alert`; si ve `NumberLit`, `Priority`. La validación del nombre de clave se difiere al análisis semántico.

## Función principal

```
pub fn parse(source: &str) -> Result<Program, Vec<VtParseError>>
```

VtParseError incluye mensaje, línea y columna (1-based), calculados desde el texto fuente. Soporta múltiples errores aunque actualmente solo retorna uno.

## Tests

10 pruebas (9 de integración + 1 unitaria):

- **5 válidos:** programa completo con 3 reglas, mínimo sin MOD0, todos los comparadores, todas las unidades, whitespace y comentarios
- **4 inválidos:** archivo vacío, llave sin cerrar, clave desconocida, valor faltante
- **1 snapshot:** salida { :#? } del programa completo verificada con insta

Todas pasan.

---

## Fase 4 — Análisis semántico

**Estado:** Mínimo (infraestructura lista, validaciones pendientes) **Crate:** vt-semantics | **Archivo:** crates/vt-semantics/src/lib.rs (41 líneas)

La función `analyze(program: &Program) -> Result<(), Vec<SemanticError>>` ejecuta una sola validación:

| Validación             | Tipo  | Mensaje                                                 |
|------------------------|-------|---------------------------------------------------------|
| program.rules no vacío | Error | “El programa debe contener al menos una regla DETECTAR” |

SemanticError tiene message, line y column — listo para reporte posicionado.

## Lo que falta

Según la especificación en docs/grammar.md, deberían implementarse:

- Validación de etiquetas contra vocabulario conocido del modelo YOLO
- Rangos de distancia (rechazar  $\leq 0$ , advertir  $> 100$ )
- Detección de prioridades duplicadas entre reglas
- Longitud máxima de alerta (200 caracteres)
- Verificación de que alerta no esté vacía

Hay un change de OpenSpec activo (phase-3-semantic-analysis) con diseño y tareas para esta fase.

---

## Fase 5 — Generación de código

**Estado:** Placeholder (infraestructura lista, lógica sin implementar) **Crate:** vt-codegen

El dispatcher (`generate(program, target)`) existe y distingue entre `Target::Python` y `Target::JavaScript`. Sin embargo, ambos backends retornan strings fijos sin recorrer el AST.

### Python (crates/vt-codegen/src/python.rs, 6 líneas)

Retorna un script con shebang y un `print("VisioTalk runtime starting...")`. El parámetro `_program` no se usa (el prefijo `_` lo marca como intencionalmente ignorado).

### JavaScript (crates/vt-codegen/src/javascript.rs, 6 líneas)

Retorna un objeto con tres hooks vacíos (`onObjectDetected`, `onDistanceThreshold`, `onAlert`) y un `export default`. Tampoco usa el AST.

**Lo que falta:** recorrer el AST y traducir reglas DETECTAR a código ejecutable. Para Python se planea usar plantillas Askama; para JS, funciones orientadas a eventos.

---

## Fase 6 — CLI y debugging

**Estado:** Funcional para frontend, backend no conectado **Crate:** `vt-cli` | **Archivo:** `crates/vt-cli/src/main.rs` (94 líneas)

### Comando

`vt compile <INPUT> [--target <TARGET>] [--debug]`

| Argumento     | Default   | Descripción                                 |
|---------------|-----------|---------------------------------------------|
| INPUT         | requerido | Archivo .vt de entrada                      |
| --target / -t | python    | Lenguaje de salida (aceptado pero ignorado) |
| --debug       | false     | Activa salida detallada fase por fase       |

### Pipeline actual

1. Lee el archivo fuente
2. Ejecuta `vt_lexer::tokenize_with_locations()`
3. Ejecuta `vt_parser::parse()`
4. Imprime el AST con `{:#?}` (modo normal) o con el debugger visual (modo `--debug`)

El flag `--target` se parsea pero el binding se llama `_target` — no se usa. El análisis semántico y la generación de código están declarados como dependencias en `Cargo.toml` pero `main()` no los invoca.

### Debugger visual (debug.rs, 113 líneas)

En modo `--debug`, el CLI imprime:

1. El código fuente completo con números de línea
2. La lista de tokens con nombre de variante
3. El AST en formato árbol usando caracteres de caja (`|—`, `└—`, `|`)

---

## Runtimes

### Desktop (runtime-desktop/)

**Estado:** Placeholder. `visio_engine.py` imprime `"VisioTalk runtime engine v0.1.0"` y sale. Existe un directorio `templates/` vacío para futuras plantillas Askama.

La arquitectura planeada (hilo de cámara → ONNX Runtime → evaluación de reglas → TTS con pyttsx3) no tiene código escrito.

## Mobile (runtime-mobile/)

**Estado:** Scaffold. Proyecto React Native 0.74+ con TypeScript, configuración de Android (`MainActivity.kt`, `MainApplication.kt`) e iOS (`AppDelegate.swift`). `App.tsx` es la pantalla por defecto de RN. No hay componentes personalizados, editor `.vt` ni integración con cámara.

---

## Documentación

Todo el directorio `docs/` está poblado y actualizado:

| Archivo                             | Contenido                                                                                                 |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <code>PRD.md</code>                 | Requisitos de producto (270 líneas): especificación DSL, arquitectura, milestones, seguridad, rendimiento |
| <code>grammar.md</code>             | Gramática EBNF formal, tabla de tokens, reglas semánticas (108 líneas)                                    |
| <code>IMPLEMENTATION_PLAN.md</code> | Plan de 9 fases con tareas atómicas (110 líneas)                                                          |
| <code>TECH_STACK.md</code>          | Stack tecnológico completo con versiones y justificaciones (287 líneas)                                   |
| <code>LEARNING_TOPICS.md</code>     | Guía de aprendizaje organizada por fase (218 líneas)                                                      |

---

## Resumen técnico

| Componente              | Estado      | Lo que hace                                                 |
|-------------------------|-------------|-------------------------------------------------------------|
| Monorepo + herramientas | Completo    | 6 crates Rust + 2 runtimes, linters configurados            |
| vt-core (AST)           | Completo    | 5 structs + 3 enums, serde                                  |
| vt-lexer                | Completo    | 18 tokens vía logos, 22 tests, API con y sin spans          |
| vt-parser               | Completo    | Gramática LALRPOP, build.rs, 10 tests, errores con posición |
| vt-semantics            | Mínimo      | 1 validación (reglas no vacías), infraestructura lista      |
| vt-codegen              | Placeholder | Dispatch Python/JS, ambos retornan strings fijos            |
| vt-cli                  | Parcial     | vt compile lee → lexea → parsea → imprime AST, debug visual |
| runtime-desktop         | Placeholder | visio_engine.py imprime versión                             |
| runtime-mobile          | Scaffold    | Proyecto RN por defecto                                     |
| Documentación           | Completa    | PRD + gramática + plan + stack                              |
| Modelos IA              | Vacío       | Directorio <code>models/</code> existe sin archivos         |

## Conteo de tests

| Crate        | Tests     | Tipo        |
|--------------|-----------|-------------|
| vt-lexer     | 22        | Unitarios   |
| vt-parser    | 9         | Integración |
| vt-parser    | 1         | Unitario    |
| vt-semantics | 1         | Unitario    |
| <b>Total</b> | <b>33</b> |             |

## Lo que sigue

1. **Integrar semántica y codegen al CLI** — `main()` ya importa ambos pero no los llama. Conectar `_target` al dispatch de codegen y ejecutar `analyze()` antes de generar código.
2. **Completar análisis semántico** — implementar las 5 validaciones descritas en `docs/grammar.md`. El change de `OpenSpec phase-3-semantic-analysis` tiene el diseño.
3. **Implementar codegen real** — recorrer el AST y producir código Python ejecutable (usando Askama) y JavaScript orientado a eventos.
4. **Runtime desktop** — implementar el pipeline de cámara + ONNX + TTS descrito en el PRD.